BRACT's

Vishwakarma Institute of Information Technology, Pune Practical Implementation Sheet

Department: CS-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No:8 Roll no: 21 PRN No: 12411682

Subject Name: Deep Learning

Name of Student: Pranay Zunzunwala

Problem Statement: Implement Pretrained BERT Model for sentiment analysis or text classification on a sample dataset.

Algorithm:

Step 1: Import the required libraries such as PyTorch, Transformers, Datasets, NumPy, Pandas, Matplotlib, and Scikit-learn.

Step 2: Load the IMDb movie-review dataset containing positive and negative reviews.

Step 3: Load the pretrained **BERT (bert-base-uncased) tokenizer** from Hugging Face.

Step 4: Tokenize the text reviews using the BERT tokenizer with padding, truncation, and a maximum sequence length of 128.

Step 5: Load the pretrained **BERT model for sequence classification** with two output classes: Positive and Negative.

Step 6: Divide the dataset into training and testing subsets.

Step 7: Configure the BERT training parameters such as learning rate, batch size, number of epochs, and weight decay.

Step 8: Fine-tune the pretrained BERT model on the IMDb training dataset.

Step 9: Evaluate the trained model using **accuracy and F1-score**.

Step 10: Generate predictions for new user-provided movie reviews.

Step 11: Display the predicted sentiment and confidence score.

Step 12: Generate a confusion matrix and classification report to analyze model performance.

Step 13: Save the fine-tuned BERT model and tokenizer for future use.

Code:

```
# Cell 1: Install required libraries
!pip -q install -U transformers datasets evaluate accelerate
print("Required libraries installed successfully.")
# Cell 2: Import required libraries
import os
import tarfile
import urllib.request
import random
import numpy as np
import pandas as pd
import torch
import matplotlib.pyplot as plt

from datasets import Dataset, DatasetDict
from transformers import (
    BertTokenizer,
    BertForSequenceClassification,
    TrainingArguments,
    Trainer
)
import evaluate

from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report
)

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

# Cell 3: Set random seeds

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
```

```
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

print("Random seed set to:", SEED)

# Cell 4: Download the IMDb dataset

DATA_URL = "https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz"
ARCHIVE_PATH = "/content/aclImdb_v1.tar.gz"
DATA_DIR = "/content/aclImdb"

if not os.path.exists(DATA_DIR):
    print("Downloading IMDb dataset...")
    urllib.request.urlretrieve(DATA_URL, ARCHIVE_PATH)
    print("Download completed.")

    print("Extracting dataset...")
    with tarfile.open(ARCHIVE_PATH, "r:gz") as tar:
        tar.extractall("/content")

    print("Extraction completed.")
else:
    print("IMDb dataset already exists.")

print("Dataset directory:", DATA_DIR)

# Cell 5: Read IMDb reviews from text files

def read_reviews(folder):
    texts = []
    labels = []

    for label_name, label_value in [("neg", 0), ("pos", 1)]:
        label_folder = os.path.join(folder, label_name)

        for filename in os.listdir(label_folder):
            if filename.endswith(".txt"):
                filepath = os.path.join(label_folder, filename)

                with open(filepath, "r", encoding="utf-8") as f:
                    texts.append(f.read())

                labels.append(label_value)

    return texts, labels
```

```
train_texts, train_labels = read_reviews(
    os.path.join(DATA_DIR, "train")
)

test_texts, test_labels = read_reviews(
    os.path.join(DATA_DIR, "test")
)

print("Total training reviews:", len(train_texts))
print("Total testing reviews:", len(test_texts))

print("\nFirst review:")
print(train_texts[0][:1000])

print("\nFirst review label:")
print("Positive" if train_labels[0] == 1 else "Negative")

# Cell 6: Create Pandas DataFrames

train_df = pd.DataFrame({
    "text": train_texts,
    "label": train_labels
})

test_df = pd.DataFrame({
    "text": test_texts,
    "label": test_labels
})

print("Training DataFrame shape:", train_df.shape)
print("Testing DataFrame shape:", test_df.shape)

print("\nTraining label distribution:")
print(train_df["label"].value_counts().sort_index())

# Cell 7: Create Hugging Face Dataset

dataset = DatasetDict({
    "train": Dataset.from_pandas(
        train_df,
        preserve_index=False
    ),
    "test": Dataset.from_pandas(
        test_df,
        preserve_index=False
```

```
    )
})

print(dataset)

# Cell 8: Select a smaller subset for faster BERT training

# This makes the assignment practical to run in Google Colab.
# 5,000 training + 1,000 testing reviews are used.

train_dataset = dataset["train"].shuffle(
    seed=SEED
).select(range(5000))

test_dataset = dataset["test"].shuffle(
    seed=SEED
).select(range(1000))

print("Training samples:", len(train_dataset))
print("Testing samples:", len(test_dataset))

# Cell 9: Load pretrained BERT tokenizer

model_name = "bert-base-uncased"

tokenizer = BertTokenizer.from_pretrained(model_name)

print("Tokenizer loaded successfully.")
print("Model:", model_name)
print("Vocabulary size:", tokenizer.vocab_size)

# Cell 10: Tokenize the reviews

def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

tokenized_train = train_dataset.map(
    tokenize_function,
    batched=True
)
```

```
tokenized_test = test_dataset.map(
    tokenize_function,
    batched=True
)

print("Tokenization completed.")
print(tokenized_train)

# Cell 11: Load pretrained BERT for sequence classification

model = BertForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)

# Label mappings
model.config.id2label = {
    0: "NEGATIVE",
    1: "POSITIVE"
}

model.config.label2id = {
    "NEGATIVE": 0,
    "POSITIVE": 1
}

print("Pretrained BERT model loaded successfully.")
print("Number of labels:", model.config.num_labels)

# Cell 12: Define evaluation metrics

accuracy_metric = evaluate.load("accuracy")
f1_metric = evaluate.load("f1")

def compute_metrics(eval_pred):
    logits, labels = eval_pred

    predictions = np.argmax(logits, axis=-1)

    accuracy = accuracy_metric.compute(
        predictions=predictions,
        references=labels
    )

    f1 = f1_metric.compute(
        predictions=predictions,
```

```
        references=labels,
        average="weighted"
    )

    return {
        "accuracy": accuracy["accuracy"],
        "f1": f1["f1"]
    }

print("Evaluation metrics configured.")

# Cell 13: Configure training parameters

training_args = TrainingArguments(
    output_dir="/content/bert_sentiment_results",

    eval_strategy="epoch",
    save_strategy="epoch",

    learning_rate=2e-5,

    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,

    num_train_epochs=2,

    weight_decay=0.01,

    logging_steps=100,

    load_best_model_at_end=True,

    report_to="none",

    fp16=torch.cuda.is_available()
)

print("Training configuration created successfully.")

# Cell 14: Create Hugging Face Trainer

trainer = Trainer(
    model=model,
    args=training_args,

    train_dataset=tokenized_train,
```

```
    eval_dataset=tokenized_test,

    processing_class=tokenizer,

    compute_metrics=compute_metrics
)

print("Trainer initialized successfully.")

# Cell 15: Train the BERT model

print("Starting BERT fine-tuning...")
print("This may take several minutes depending on the Colab GPU.")

train_result = trainer.train()

print("\nTraining completed successfully!")

# Cell 16: Evaluate the model

evaluation_results = trainer.evaluate()

print("BERT Evaluation Results")
print("=" * 40)

for key, value in evaluation_results.items():
    print(f"{key}: {value}")

# Cell 17: Generate predictions on the test dataset

prediction_output = trainer.predict(tokenized_test)

predicted_labels = np.argmax(
    prediction_output.predictions,
    axis=1
)

actual_labels = prediction_output.label_ids

print("Prediction completed.")
print("Number of predictions:", len(predicted_labels))

# Cell 18: Classification report

print("Classification Report")
print("=" * 60)
```

```
print(
    classification_report(
        actual_labels,
        predicted_labels,
        target_names=["Negative", "Positive"]
    )
)

# Cell 19: Confusion matrix

cm = confusion_matrix(
    actual_labels,
    predicted_labels
)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Negative", "Positive"]
)

disp.plot()
plt.title("BERT Sentiment Analysis - Confusion Matrix")
plt.show()

# Cell 20: Function for custom sentiment prediction

def predict_sentiment(text):
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=128
    )

    device = model.device

    inputs = {
        key: value.to(device)
        for key, value in inputs.items()
    }

    model.eval()

    with torch.no_grad():
```

```
    outputs = model(**inputs)

  probabilities = torch.softmax(
    outputs.logits,
    dim=-1
  )

  prediction = torch.argmax(
    probabilities,
    dim=-1
  ).item()

  labels = {
    0: "Negative",
    1: "Positive"
  }

  sentiment = labels[prediction]
  confidence = probabilities[0][prediction].item()

  return sentiment, confidence

# Cell 21: Test a positive review

review = "This movie was absolutely fantastic. The acting and story were excellent."

sentiment, confidence = predict_sentiment(review)

print("Review:")
print(review)

print("\nPredicted Sentiment:", sentiment)
print("Confidence:", round(confidence * 100, 2), "%")

# Cell 22: Test a negative review

review = "This movie was terrible and boring. I did not enjoy it at all."

sentiment, confidence = predict_sentiment(review)

print("Review:")
print(review)

print("\nPredicted Sentiment:", sentiment)
print("Confidence:", round(confidence * 100, 2), "%")
```

```
# Cell 23: Test multiple custom reviews

reviews = [
    "I really enjoyed this movie. It was amazing!",
    "The movie was boring and poorly made.",
    "Excellent acting and a wonderful storyline.",
    "I regret watching this movie. It was terrible.",
    "The movie was okay but nothing special."
]

print("BERT SENTIMENT ANALYSIS")
print("=" * 70)

for review in reviews:
    sentiment, confidence = predict_sentiment(review)

    print("\nReview:", review)
    print("Sentiment:", sentiment)
    print("Confidence:", round(confidence * 100, 2), "%")

# Cell 24: Save the trained model

SAVE_PATH = "/content/bert_sentiment_model"

trainer.save_model(SAVE_PATH)
tokenizer.save_pretrained(SAVE_PATH)

print("Model saved successfully.")
print("Location:", SAVE_PATH)

# Cell 25: Load the saved model

loaded_tokenizer = BertTokenizer.from_pretrained(
    SAVE_PATH
)

loaded_model = BertForSequenceClassification.from_pretrained(
    SAVE_PATH
)

print("Saved BERT model loaded successfully.")

# Cell 26: Test the saved model

def predict_with_saved_model(text):
```

```
    device = torch.device(
        "cuda" if torch.cuda.is_available() else "cpu"
    )

    loaded_model.to(device)
    loaded_model.eval()

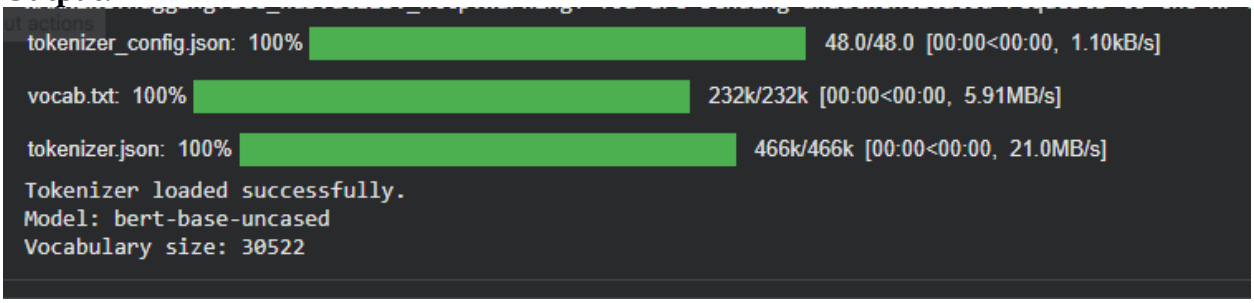
    inputs = loaded_tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=128
    )
    inputs = {
        key: value.to(device)
        for key, value in inputs.items()
    }
    with torch.no_grad():
        outputs = loaded_model(**inputs)

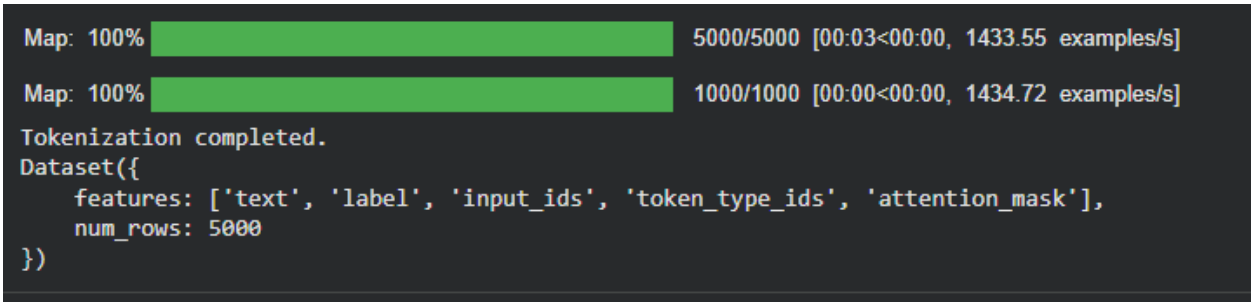
    probabilities = torch.softmax(
        outputs.logits,
        dim=-1
    )
    prediction = torch.argmax(
        probabilities,
        dim=-1
    ).item()
    labels = {
        0: "Negative",
        1: "Positive"
    }
    print("Text:", text)
    print("Predicted Sentiment:", labels[prediction])
    print(
        "Confidence:",
        round(
            probabilities[0][prediction].item() * 100,
            2
        ),
        "%"

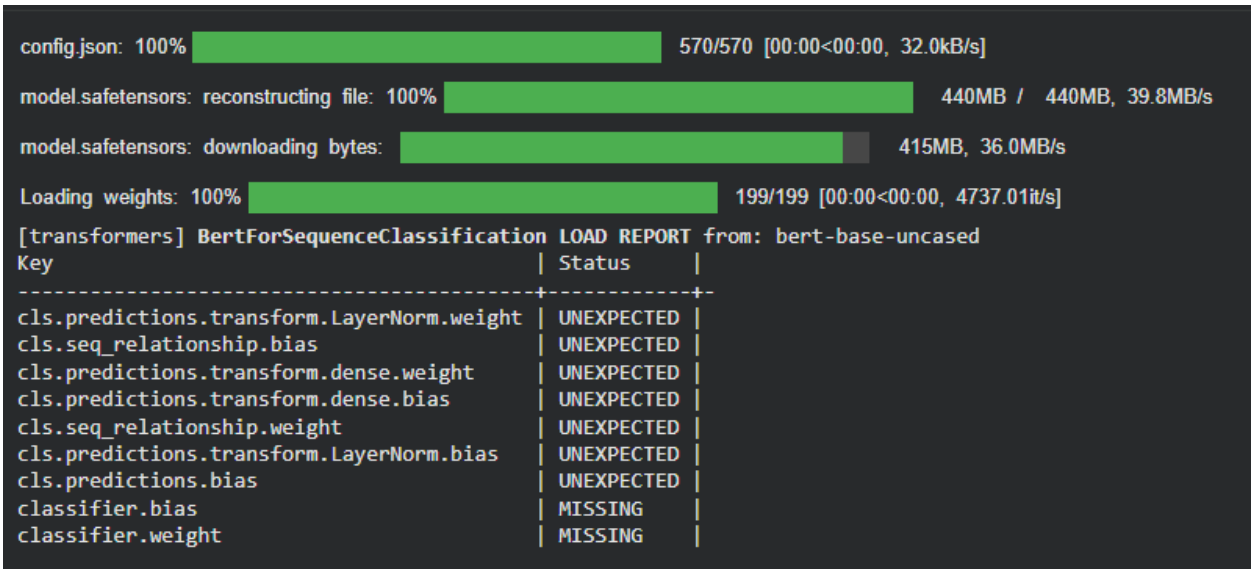
predict_with_saved_model(
    "The movie was wonderful and I loved every moment of it."
```

)

Output:


tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.10kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 5.91MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 21.0MB/s]
Tokenizer loaded successfully.
Model: bert-base-uncased
Vocabulary size: 30522


Map: 100% 5000/5000 [00:03<00:00, 1433.55 examples/s]
Map: 100% 1000/1000 [00:00<00:00, 1434.72 examples/s]
Tokenization completed.
Dataset({
features: ['text', 'label', 'input_ids', 'token_type_ids', 'attention_mask'],
num_rows: 5000
})


config.json: 100% 570/570 [00:00<00:00, 32.0kB/s]
model.safetensors: reconstructing file: 100% 440MB / 440MB, 39.8MB/s
model.safetensors: downloading bytes: 415MB, 36.0MB/s
Loading weights: 100% 199/199 [00:00<00:00, 4737.01it/s]
[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key | Status |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

```
Starting BERT fine-tuning...
This may take several minutes depending on the Colab GPU.
```

[1250/1250 02:28, Epoch 2/2]

Epoch	Training Loss	Validation Loss	Accuracy	F1
1	0.385016	0.333172	0.867000	0.866960
2	0.210304	0.467119	0.873000	0.872939

```
Writing model shards: 100%  1/1 [00:07<00:00,  7.17s/it]
Writing model shards: 100%  1/1 [00:03<00:00,  3.59s/it]
```

[125/125 00:02]

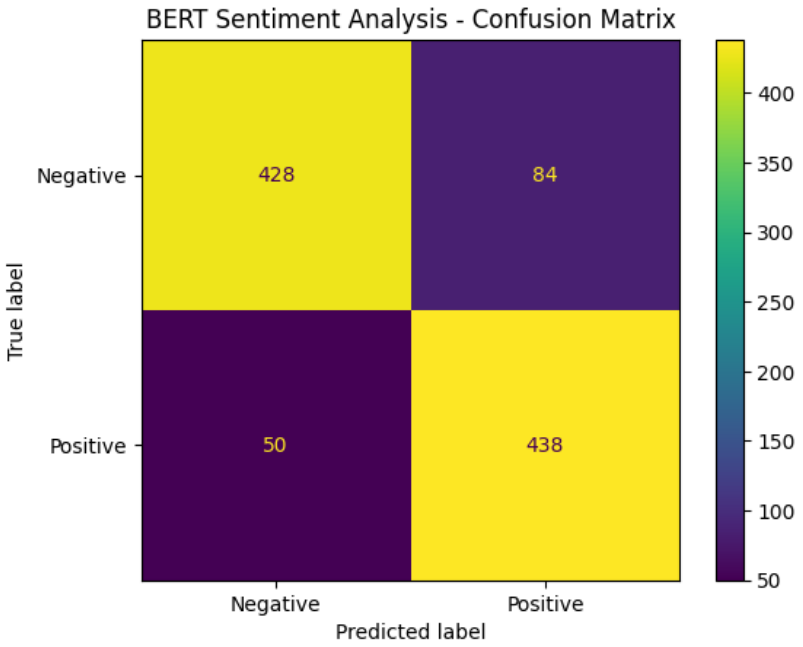
Training Loss	Validation Loss	Epoch	Accuracy	F1
0.210304	0.333843	2	0.866000	0.865954

```
BERT Evaluation Results
========================================
eval_loss: 0.33384305238723755
eval_accuracy: 0.866
eval_f1: 0.8659544354435443
```

```
Classification Report
============================================================
              precision    recall  f1-score   support

    Negative       0.90      0.84      0.86       512
    Positive       0.84      0.90      0.87       488

    accuracy                           0.87      1000
   macro avg       0.87      0.87      0.87      1000
weighted avg       0.87      0.87      0.87      1000
```




```
BERT SENTIMENT ANALYSIS
================================================================

Review: I really enjoyed this movie. It was amazing!
Sentiment: Positive
Confidence: 98.69 %

Review: The movie was boring and poorly made.
Sentiment: Negative
Confidence: 98.62 %

Review: Excellent acting and a wonderful storyline.
Sentiment: Positive
Confidence: 98.71 %

Review: I regret watching this movie. It was terrible.
Sentiment: Negative
Confidence: 97.79 %

Review: The movie was okay but nothing special.
Sentiment: Positive
Confidence: 87.67 %
```

```
Text: The movie was wonderful and I loved every moment of it.
Predicted Sentiment: Positive
Confidence: 98.15 %
```